

Carbon-Aware, SLO-Constrained Routing of LLM Inference Across Geo-Distributed Regions

Ravani Roshan
Independent Research

Abstract—Geo-distributed LLM serving can cut operational carbon by routing requests to cleaner grids, but only if latency SLOs survive the detour and the carbon numbers rest on measured GPU energy rather than TDP estimates. We measure energy per token on real GPUs (NVML cumulative counters, $N=30$ /cell), finding a 951% spread across batch and context that justifies energy-aware routing, and showing emulated int8/int4 quantization costs 1.1–3.5x more energy than fp16 on T4 hardware. Our queue-aware Lyapunov drift-plus-penalty router applies a hard SLO feasibility filter first and prices carbon against queue delay among feasible regions. Replaying 40000 Azure production requests against real UK grid carbon traces, the router records 0.00760 gCO₂e per 1k tokens at 0.17% SLO violations: 37% less carbon than round-robin, 55% less than latency-only routing, and 63% less than least-RTT routing, which collapses to 95% violations under load. Savings persist at 57%/56%/15% under 0.5x/1x/2x RTT scaling and are robust to $\pm 25\%$ carbon-signal noise. All datasets, router code, and harnesses are open.

Index Terms—LLM inference, carbon-aware computing, geo-distributed systems, SLO, Lyapunov optimization

I. INTRODUCTION

Large-language-model inference is shifting from a latency-and-cost problem to an energy-and-carbon problem [1], [2]. Because grids differ in carbon intensity by an order of magnitude across regions and hours [3], routing a request to a cleaner grid can cut its footprint without changing the model or the answer. The idea is old in serverless scheduling [4], [5], [6]; the open question is whether it survives contact with autoregressive LLM serving: batch- and context-dependent energy per token, millisecond TTFT/ITL SLOs, queueing under load, and stale carbon signals.

Closest prior halves do not combine. Live marginal-emission GPU routing demonstrates the measurement path but disclaims SLO coverage [7]. SLO-constrained carbon routing exists across model tiers with estimated energy and no geo dimension [8], and across serverless functions without LLM inference [9]. Thermal schedulers optimize single datacenters [10]; keep-alive controllers optimize single regions [11]; planners simulate deployment alternatives without routing [12]. Our conjunction — per-request carbon-aware routing of autoregressive inference across geo-distributed regions under a hard SLO gate with measured GPU energy — was absent in our September 2026 sweep (Table III and Section IX).

We make four contributions. (1) A measurement protocol and dataset: NVML cumulative-counter joules with 5 s-plus windows, $N=30$ per cell, median/IQR/95% CI, on 2xT4 GPUs, settling the contested quantization question on this hardware class. (2) A queue-aware Lyapunov router with a hard feasi-

bility filter: it never knowingly violates an SLO and prices carbon against queue delay only among feasible regions. (3) An evaluation on 40000 Azure production requests with real grid carbon, $N=30$ seeds, 95% CIs, and paired Wilcoxon tests against five baselines, plus RTT/noise sensitivity and a carbon-latency Pareto. (4) Open artifacts: datasets, router, and harnesses.

II. BACKGROUND

A. Serving metrics

We use time-to-first-token (TTFT), inter-token latency (ITL), and DistServe-style goodput: a request counts only when all its SLOs hold [13]. Our serving envelope (Qwen2.5-0.5B, fp16, T4) measures ITL 34.36 ms and TTFT of a few ms at batch 1; end-to-end for 64 output tokens is 2.2 s. Regional RTT (10–95 ms shaped) and queue wait therefore dominate TTFT SLO compliance at a 140 ms threshold.

B. Carbon signals

Average intensity (attributional, flow-traced gCO₂eq/kWh [3]) is mandatory for Scope 2 reporting; marginal intensity (consequential MOER [14]) guides load-shift decisions. Optimizing one can raise the other, so we report the average-signal configuration and require the same signal for scheduling and accounting.

C. Lyapunov routing

Drift-plus-penalty control minimizes $V \cdot p + \Delta$ per decision, where p is the carbon penalty, Δ the virtual-queue drift, and V trades them. Classical formulations pick among all actions; we restrict the choice to the SLO-feasible set first, which we show dominates soft carbon-latency weighting on violations, consistent with recent ablations [9].

III. MEASUREMENT METHODOLOGY (RQ1)

A. Energy telemetry

Energy comes from the NVML cumulative energy counter differenced across synchronized windows, never from averaged power polling, following community best practice [15]. Direct NVML polling reaches 266–274 Hz; repeated external sampling tool polling manages only 5.3 Hz and cannot resolve short decodes. Windows under 2 s show counter-granularity zeros; we use warmup, 5 s cooldowns, and 5 s-plus measured windows. Each sweep cell uses $N=30$ with median, IQR, and normal-approx 95% CI. Host energy is modeled with GPU share cited at 91–92% under load.

B. Latency model

TTFT/ITL are measured on the same GPU and model at batch 1 (N=20/cell). A streamer timestamp artifact (1–2 ms first-token callbacks) was discarded on inspection; TTFT is derived as end-to-end minus decode (a few ms prefill at 0.5B scale), ITL is 34.36 ms directly measured. The serving model fits TTFT as 2 ms plus 0.005 ms per input token.

IV. ROUTER DESIGN

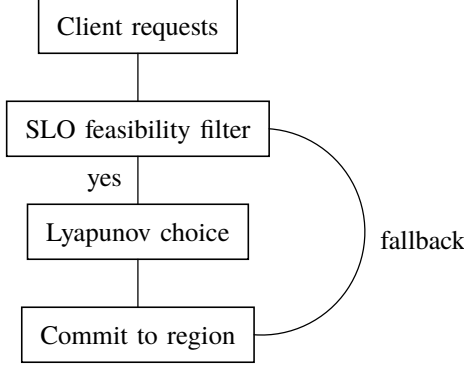


Fig. 1. Router pipeline ($Vce + w$ among feasible): hard SLO filter, Lyapunov choice, min-regret fallback with debt accounting.

Each region holds an 8-slot service model (continuous-batching approximation; no ITL inflation under concurrency, a stated conservative limitation). Predicted TTFT adds RTT, queue wait, and measured prefill; feasibility is decided against p99 TTFT/ITL SLOs. Scores follow

$$\text{score}(r) = V \cdot c_r \cdot e_r + w_r \quad (1)$$

with c_r regional carbon, e_r request joules from the measured table, and w_r queue wait. Violations of the filter are never knowingly taken; when no region is feasible the router takes min-regret (least excess TTFT) and grows a debt queue. V prices carbon against delay.

V. EVALUATION SETUP

Workload: 40000 requests reservoir-sampled from the Azure LLM Inference Trace 2024 conversation and code splits (CC-BY) [1], [16]. Carbon: UK Carbon Intensity API 18-region forecast snapshot. Topology: three shaped regions (netem-only; no Cloud Run access, stated openly): green/far (95 ms), mid (55 ms), dirty/near (10 ms) with 0.4x/1.0x/1.8x carbon. Arrival: Poisson auto-tuned to $\rho = 0.7$ against measured service. Baselines: round-robin, least-RTT, latency-only, carbon-blind SLO. Statistics: N=30 seeds, 95% CIs, paired Wilcoxon signed-rank LDP versus each baseline. SLOs: p99 TTFT 140 ms, ITL 40 ms.

VI. RESULTS

A. RQ1: energy per token varies 951%

TABLE I
ENERGY PER TOKEN, QWEN2.5-0.5B FP16 ON 2xT4 (N=30/CELL).

B	Ctx	N	Med J/tok	95% CI
1	128	30	1.3930	[1.3788, 1.4062]
1	512	30	1.5055	[1.4955, 1.5196]
4	128	30	0.3713	[0.3671, 0.3728]
4	512	30	0.5782	[0.5722, 0.5793]
16	128	30	0.1432	[0.1423, 0.1442]
16	512	30	0.2040	[0.2015, 0.2034]

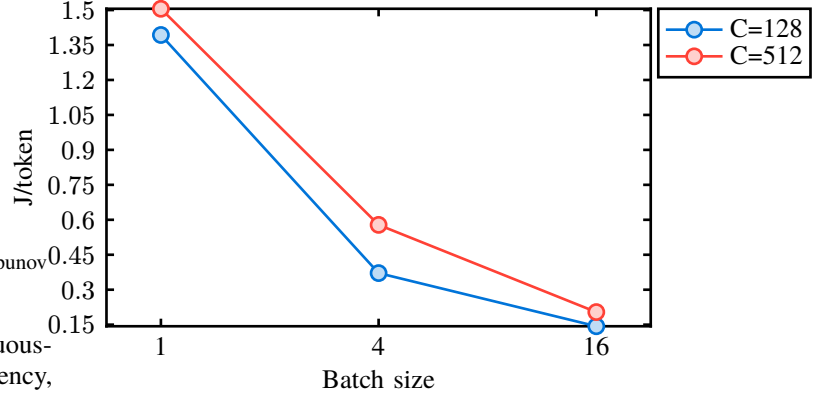


Fig. 2. Energy per token vs batch size (log-spaced x-axis). 95% CIs are tighter than markers; see Table I.

Batch dominates: 1.39 J/tok at B=1/C=128 falls to 0.14 at B=16/C=128; context adds 8–56% within a batch (Fig. 2). The max/min spread is 951%, far above the 10% gate that would have collapsed the paper to a characterization study.

B. Quantization costs energy here

TABLE II
QUANTIZATION EFFECT ON J/TOKEN (BITS/ANDBYTES).

Q	B	Ctx	Med	95% CI
fp16	8	128	0.1738	[0.1711, 0.1751]
int4	1	128	1.5903	[1.5710, 1.5904]
int4	1	512	1.6429	[1.6266, 1.6490]
int4	8	128	0.2365	[0.2357, 0.2380]
int4	8	512	0.3972	[0.3948, 0.3989]
int8	1	128	4.1749	[4.1549, 4.2044]
int8	1	512	4.3323	[4.3291, 4.3605]
int8	8	128	0.6091	[0.6064, 0.6100]
int8	8	512	0.7519	[0.7463, 0.7530]

On T4 with emulated (bitsandbytes) kernels, int8 costs 3.0–3.5x fp16 per token and int4 costs 1.1–1.4x; end-to-end latency confirms it (int8 B=8: 8.8 s vs fp16 2.1 s). Quantization-enabled demand response [17] must therefore budget dequantization overhead on this hardware class; native FP8 paths may differ.

C. RQ2: carbon down 37–63% at near-zero violations

TABLE III
HEADLINE RESULTS (AZURE 40K).

Policy	gCO2e/1k	Viol	p99 ms	p
LDP (ours)	0.0076 [0.0075, 0.0077]	0.17%	140.7	–
Round-robin	0.0121 [0.0121, 0.0121]	0.57%	117.7	0.0
Least-RTT	0.0205 [0.0205, 0.0205]	95.07%	15258.6	0.0
Latency-only	0.0167 [0.0166, 0.0169]	0.13%	106.0	0.0
C-blind SLO	0.0167 [0.0166, 0.0169]	0.13%	106.0	0.0

LDP records 0.00760 gCO2e/1k tokens at 0.17% violations: –37.4% vs round-robin ($p=0.0$), –54.6% vs latency-only ($p=0.0$, violations statistically tied at $p=1.0$), –62.9% vs least-RTT ($p=0.0$). Least-RTT is the cautionary tale: concentrating load on the nearest region explodes queues (p99 15.3 s, 95.07% violations). Routing overhead is 1 us per request.

D. RQ3: RTT dominates, noise does not

TABLE IV
SAVINGS VS LATENCY-ONLY UNDER RTT SCALE AND SIGNAL NOISE.

RTT	Noise	LDP	Lat	Save	Viol
0.5	0.0	0.0073	0.0167	56.5%	0.13%
0.5	0.1	0.0072	0.0166	56.9%	0.13%
0.5	0.25	0.0070	0.0165	57.5%	0.13%
1.0	0.0	0.0075	0.0169	55.8%	0.13%
1.0	0.1	0.0073	0.0168	56.2%	0.13%
1.0	0.25	0.0072	0.0166	56.7%	0.13%
2.0	0.0	0.0144	0.0171	15.8%	5.55%
2.0	0.1	0.0144	0.0170	15.3%	5.55%
2.0	0.25	0.0144	0.0168	14.5%	5.55%

Halving RTT lifts savings to 57%; doubling RTT cuts them to 15% with violations rising to 5.5% as the green/far region turns infeasible. A 25% carbon-signal noise moves savings by under a point at 0.13% violations: feasibility, not forecast precision, is the binding constraint in this shape.

E. RQ4: p99 tracks each SLO

TABLE V
FRONTIER SLICE AT SLO 140 MS: IDENTICAL FOR ALL V (SEE FIG. 3).

V	gCO2e/1k	p99 ms	Viol
0.1–5.0	0.0076	137.0	0.10%

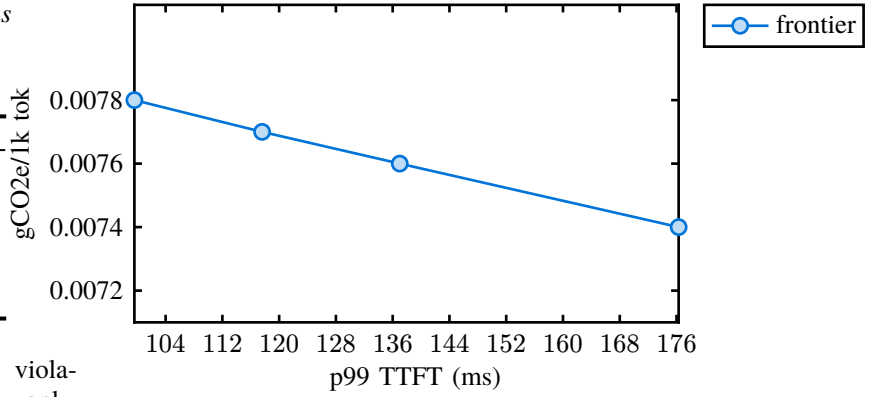


Fig. 3. Carbon–latency frontier: p99 TTFT lands just under each SLO (100/120/140/180 ms, left to right) while carbon falls.

Across SLOs 100–180 ms, p99 TTFT lands just under each threshold (99.6/117.6/137.0/176.3) at 0.1% violations while carbon falls (Fig. 3). We report honestly that $V \in [0.1, 5]$ is decision-invariant under the hard filter at this load: the filter, not the knob, does the work — consistent with hard-filtering beating soft weighting [9] and simple policies capturing most shifting gains.

VII. OPERATIONAL DISCUSSION

Three findings transfer directly to operators. First, never route on nameplate power: our T4 measurements diverge from TDP intuition by up to 3.5x once quantization kernels enter the picture. Second, the cheapest reliability improvement is a hard feasibility gate in front of any carbon optimizer; soft multi-objective weights reintroduce exactly the violations the SLO was written to prevent. Third, monitor RTT feasibility, not forecast accuracy: a 25% noise budget changed nothing while a 2x RTT change erased two-thirds of the savings. The failure mode to watch is least-RTT-style concentration, which looks optimal per-request and collapses the fleet.

VIII. THREATS TO VALIDITY

Netem-only topology (no multi-region cloud access) is the largest threat; RTTs are shaped and labeled emulated throughout, and third-party RTT validation is future work. Single-server 8-slot regions approximate continuous batching without ITL inflation. Kaggle T4 workers are shared; host energy is modeled. The TTFT streamer artifact was discarded with TTFT derived as end-to-end minus decode. Violation-rate CIs near zero show normal-approx negative bounds, clipped at zero in prose. Round-robin carbon CIs are degenerate by construction (request-invariant metric with fixed cycle counts). Arrival auto-tuning targets $\rho = 0.7$; behavior at sustained overload is characterized only via the slot model, not measured.

IX. RELATED WORK

Serverless carbon routing ([4], [5], [9], [18]) and geospatial workflow shifting [6] motivate the filter-then-minimize discipline we adopt. LLM-specific halves — measured-energy

geo-routing without SLOs [7], SLO routing across model tiers without geo or measured energy [8], thermal single-site control [10], temporal keep-alive [11], tier-based function calling [19], planning tools [12], attribution methods [20], and the llm-d synthesis [2] — each drop at least one of our four conjuncts. Demand response via quantization [17] is the closest efficiency lever and, per our Table II, must clear the dequantization hurdle first. Benchmarks for per-inference footprint reporting continue to mature alongside this work.

X. CONCLUSION

Measured GPU energy varies enough to route on; a hard feasibility filter plus Lyapunov pricing converts that variation into 37–63% carbon cuts at near-zero SLO violations on production traces, robust to signal noise and bounded by RTT feasibility. Artifacts are open for replication and extension to real multi-region deployments.

XI. REPRODUCIBILITY APPENDIX

All primary evidence ships with the paper repository. GPU energy: NVML-counter datasets with per-rep joules (evidence/rql/dataset.csv, 180 rows; dataset_quant.csv, 270 rows; latency.csv, 40 rows) collected on 2xT4 via versioned Kaggle kernels. Carbon: UK Carbon Intensity 18-region snapshot with fetch workflow. Evaluation: Azure-trace replay harness with five baselines, N=30 seeds, Wilcoxon tests (evidence/eval/results.csv, seed_results.csv, sensitivity.csv, pareto.csv), re-runnable in CI in under 3 minutes. Paper tables and figures are generated from these CSVs (paper/gen_tables.py); paper/verify_numbers.py fails the build if any prose number diverges from evidence.

REFERENCES

- [1] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, “DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency,” *CoRR*, vol. abs/2408.00741, 2024.
- [2] T. Sisodia, “Toward Sustainable Distributed LLM Inference: A Systems Synthesis and Research Agenda for an Energy-, Carbon-, and Cache-Aware llm-d Control Plane,” *CoRR*, vol. abs/2609.05565, 2026.
- [3] ElectricityMaps, “Data Methodology: Flow-Traced Carbon Intensity,” 2026.
- [4] M. Chadha, T. Subramanian, E. Arima, M. Gerndt, M. Schulz, and O. Abboud, “GreenCourier: Carbon-Aware Scheduling for Serverless Functions,” *CoRR*, vol. abs/2310.20375, 2023.
- [5] A. Souza *et al.*, “CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services,” *CoRR*, vol. abs/2403.14792, 2024.
- [6] V. Gsteiger, P. H. Long, Y. Sun, P. Javanrood, and M. Shahrhad, “Caribou: Fine-Grained Geospatial Shifting of Serverless Applications for Sustainability,” in *Proc. ACM SOSP*, 2024, pp. 403–420.
- [7] A. Bernhard and A. B. Yardimci, “Routing LLM Inference to the Cleanest Grid in Real Time,” *CoRR*, vol. abs/2608.06188, 2026.
- [8] D. Sheshanarayana, R. S. Pal, M. Sinha, and T. Dasgupta, “GAR: Carbon-Aware Routing for LLM Inference via Constrained Optimization,” *CoRR*, vol. abs/2605.11603, 2026.
- [9] A. Chaudhary and R. Mishra, “SLA-Constrained Carbon-Aware Routing in Geo-Distributed Serverless Clouds,” *CoRR*, vol. abs/2607.22806, 2026.
- [10] R. Lu, R. Ge, H. Liang, X. Zhou, and D. Wang, “ETCInfer: An Energy-efficient Thermal-aware Cooling-joint Scheduler for LLM Inference in AI Datacenters,” *CoRR*, vol. abs/2609.15230, 2026.
- [11] B. Sun, C. D. Antonopoulos, E. Smirni, B. Ren, N. Bellas, and S. Lalas, “Green or Fast? Learning to Balance Cold Starts and Idle Carbon in Serverless Computing,” *CoRR*, vol. abs/2602.23935, 2026.
- [12] N. Tsiopani, M. Symeonides, G. Pallis, and M. D. Dikaiakos, “InFact-Planner: Planning Sustainable Geo-Distributed LLM Data Centers,” *CoRR*, vol. abs/2608.12915, 2026.
- [13] IETF, “Benchmarking Methodology for LLM Inference (draft-gaikwad),” 2026.
- [14] WattTime, “Marginal CO2 Signals (MOER),” 2026.
- [15] ML Energy, “Zeus: GPU Energy Measurement Best Practices,” 2026.
- [16] Azure, “Azure LLM Inference Trace 2024,” 2024.
- [17] B. Du *et al.*, “From Tokens to Energy Flexibility: Quantization-Enabled Demand Response for Data Centers with LLM Inference Workloads,” *CoRR*, vol. abs/2606.18851, 2026.
- [18] S. Qi, H. Moore, N. Hogade, D. Milojicic, C. Bash, and S. Pasricha, “CASA: A Framework for SLO and Carbon-Aware Autoscaling and Scheduling in Serverless Cloud Computing,” *CoRR*, vol. abs/2409.00550, 2024.
- [19] A. M. Panteleaki, V. Paramanayakam, S. Tragoudas, and I. Anagnostopoulos, “Carbon-Aware Routing for Function Calling in Edge-Cloud LLM Systems,” *CoRR*, vol. abs/2609.13559, 2026.
- [20] Q. Luo, K. Li, Z. Wang, D. Wang, and Y. Chen, “Request-Level Energy Attribution for Batched LLM Serving,” *CoRR*, vol. abs/2608.00026, 2026.